Sampling Methods for Retail Sales Estimation

DATA 407 Project

AUTHOR
Dia Toredi (16967911)

AFFILIATION
UBC Okanagan

PUBLISHED
March 31, 2026

Introduction

In many real-world applications, collecting data from an entire population is impractical due to time, cost, and logistical constraints. As a result, sampling methods are widely used to estimate population parameters. In the context of retail analytics, companies often rely on sampled data to estimate average sales, customer behavior, and demand patterns.

The purpose of this project is to evaluate how different sampling techniques affect the accuracy of estimating average retail sales. Specifically, we compare **simple random sampling (SRS)**, **stratified sampling**, and **cluster sampling** using a simulated retail dataset. Understanding the strengths and weaknesses of these methods is essential for making reliable statistical inferences.

Research Questions:

1. Which sampling method provides the most accurate (lowest bias) estimate of the population mean?
2. Which sampling method provides the most precise (lowest variance) estimates?
3. How does the choice of sampling design affect the reliability of retail sales estimates?

This study is motivated by the need for efficient and accurate data collection strategies in large-scale retail environments where surveying all stores or transactions is prohibitively expensive.

Methods

Three sampling methods are evaluated in this analysis:

Simple Random Sampling (SRS): Each observation has an equal probability of selection. This method is straightforward and produces unbiased estimates, but may not efficiently capture subpopulation differences.

- Estimator:

$$\bar{y}_{SRS} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

- Variance:

$$\mathrm{Var}(\bar{y}_{SRS}) = \frac{S^2}{n}\left(1 - \frac{n}{N}\right)$$

Stratified Random Sampling: The population is divided into homogeneous strata (e.g., regions), and independent random samples are drawn from each stratum. This reduces variance when strata differ in their means.

- Estimator:

$$\bar{y}_{str} = \sum_{h=1}^{H} W_h \bar{y}_h$$

  where

$$W_h = \frac{N_h}{N}$$

- Variance:

$$\mathrm{Var}(\bar{y}_{str}) = \sum_{h=1}^{H} W_h^2 \frac{S_h^2}{n_h}\left(1 - \frac{n_h}{N_h}\right)$$

Cluster Sampling: The population is divided into clusters (e.g., stores or regions), and a random sample of clusters is selected. All units within selected clusters are observed. This is cost-effective but can increase variance if clusters are heterogeneous internally but similar to each other.

- Estimator:

$$\bar{y}_{clus} = \frac{\sum_{i=1}^{m} M_i \bar{y}_i}{\sum_{i=1}^{m} M_i}$$

  for unequal cluster sizes

Evaluation Metrics:

- **Bias:**

$$\mathrm{Bias} = E[\hat{\theta}] - \theta$$

- **Variance:**

$$\mathrm{Var}(\hat{\theta})$$

- **Mean Squared Error:**

$$\mathrm{MSE} = \mathrm{Bias}^2 + \mathrm{Variance}$$

- **95% Confidence Interval Coverage:** Proportion of intervals containing the true mean

These methods were chosen because they represent the most common sampling designs used in practice and allow direct comparison of the bias-variance tradeoff.

Simulation

A Monte Carlo simulation is conducted to evaluate each sampling method under controlled conditions. By drawing repeated samples (1000 replications), we can reliably estimate bias, variance, and MSE for each method.

Population Setup

```r
library(ggplot2)
library(dplyr)
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
set.seed(123)

# Create realistic retail population
N <- 5000

# Create regions with different characteristics
region <- sample(c("North", "South", "West"), N, replace = TRUE,
                 prob = c(0.3, 0.4, 0.3))

# Sales vary by region (realistic heterogeneity)
sales <- numeric(N)
sales[region == "North"] <- rlnorm(sum(region == "North"), meanlog = 6.2, sdlog = 0.4)
sales[region == "South"] <- rlnorm(sum(region == "South"), meanlog = 5.8, sdlog = 0.6)
sales[region == "West"] <- rlnorm(sum(region == "West"), meanlog = 6.0, sdlog = 0.5)

# Add store ID for cluster sampling
store_id <- sample(1:50, N, replace = TRUE)

population <- data.frame(region, store_id, sales)

# Population parameters
true_mean <- mean(population$sales)
true_sd <- sd(population$sales)

cat("Population size:", N, "\n")
```

```
Population size: 5000
```

```r
cat("True population mean: $", round(true_mean, 2), "\n")
```

```
True population mean: $ 454.34
```

```r
cat("True population SD: $", round(true_sd, 2), "\n")
```

```
True population SD: $ 248.87
```

Population Structure

```r
# Examine population by region
population %>%
  group_by(region) %>%
  summarise(
    N = n(),
    Proportion = n()/nrow(population),
    Mean = mean(sales),
    SD = sd(sales)
  )
```

region	N	Proportion	Mean	SD
<chr>	<int>	<dbl>	<dbl>	<dbl>
North	1487	0.2974	534.1796	220.6862
South	1992	0.3984	393.8959	259.3194
West	1521	0.3042	455.4342	238.6923

3 rows

The regions have different sample sizes and different mean sales, making stratified sampling potentially advantageous.

Monte Carlo Simulation

```r
set.seed(456)
n_sims <- 1000
sample_size <- 200

# Storage for results
results_srs <- numeric(n_sims)
results_strat <- numeric(n_sims)
results_cluster <- numeric(n_sims)

se_srs <- numeric(n_sims)
se_strat <- numeric(n_sims)
se_cluster <- numeric(n_sims)

# Region proportions for stratified sampling
region_props <- population %>%
  group_by(region) %>%
  summarise(N_h = n(), W_h = n()/nrow(population))

for (i in 1:n_sims) {

  # --- Simple Random Sampling ---
  srs_sample <- population[sample(1:N, sample_size, replace = FALSE), ]
  results_srs[i] <- mean(srs_sample$sales)
  se_srs[i] <- sd(srs_sample$sales) / sqrt(sample_size)

  # --- Stratified Random Sampling (Proportional Allocation) ---
  strat_sample <- population %>%
    group_by(region) %>%
    sample_n(size = round(sample_size * first(
      region_props$W_h[region_props$region == first(region)]
    ))) %>%
    ungroup()

  # Stratified mean (weighted by population proportions)
  strat_means <- strat_sample %>%
    group_by(region) %>%
    summarise(ybar_h = mean(sales), s_h = sd(sales), n_h = n()) %>%
    left_join(region_props, by = "region")

  results_strat[i] <- sum(strat_means$W_h * strat_means$ybar_h)
  se_strat[i] <- sqrt(sum(strat_means$W_h^2 * strat_means$s_h^2 / strat_means$n_h))

  # --- Cluster Sampling (Random sample of stores) ---
  n_clusters <- 10
  selected_stores <- sample(unique(population$store_id), n_clusters)
  cluster_sample <- population[population$store_id %in% selected_stores, ]

  results_cluster[i] <- mean(cluster_sample$sales)
  se_cluster[i] <- sd(cluster_sample$sales) / sqrt(nrow(cluster_sample))
}
```

Simulation Results

```r
# Calculate performance metrics
calc_metrics <- function(estimates, true_value, std_errors) {
  bias <- mean(estimates) - true_value
  variance <- var(estimates)
  mse <- bias^2 + variance

  # CI coverage
  lower <- estimates - 1.96 * std_errors
  upper <- estimates + 1.96 * std_errors
  coverage <- mean(lower <= true_value & upper >= true_value)

  return(c(Bias = bias, Variance = variance, MSE = mse,
           RMSE = sqrt(mse), Coverage = coverage))
}

metrics_srs <- calc_metrics(results_srs, true_mean, se_srs)
metrics_strat <- calc_metrics(results_strat, true_mean, se_strat)
metrics_cluster <- calc_metrics(results_cluster, true_mean, se_cluster)

# Summary table
summary_table <- data.frame(
  Method = c("Simple Random", "Stratified", "Cluster"),
  Mean_Estimate = c(mean(results_srs), mean(results_strat), mean(results_cluster)),
  Bias = c(metrics_srs["Bias"], metrics_strat["Bias"], metrics_cluster["Bias"]),
  Variance = c(metrics_srs["Variance"], metrics_strat["Variance"], metrics_cluster["Vari
  RMSE = c(metrics_srs["RMSE"], metrics_strat["RMSE"], metrics_cluster["RMSE"]),
  CI_Coverage = c(metrics_srs["Coverage"], metrics_strat["Coverage"], metrics_cluster["C
)

print(summary_table)
```

```
         Method Mean_Estimate        Bias Variance      RMSE CI_Coverage
1 Simple Random      453.4224 -0.91385531 301.1352 17.377293       0.939
2    Stratified      453.8552 -0.48108032 291.0589 17.067229       0.938
3       Cluster      454.2560 -0.08026118  45.3770  6.736723       0.975
```

Interpretation of Simulation Results

Metric	SRS	Stratified	Cluster	Best
Bias	-0.91	-0.48	-0.08	SRS
Variance	301.14	291.06	45.38	Cluster
RMSE	17.38	17.07	6.74	Cluster
95% CI Coverage	93.9%	93.8%	97.5%	Stratified

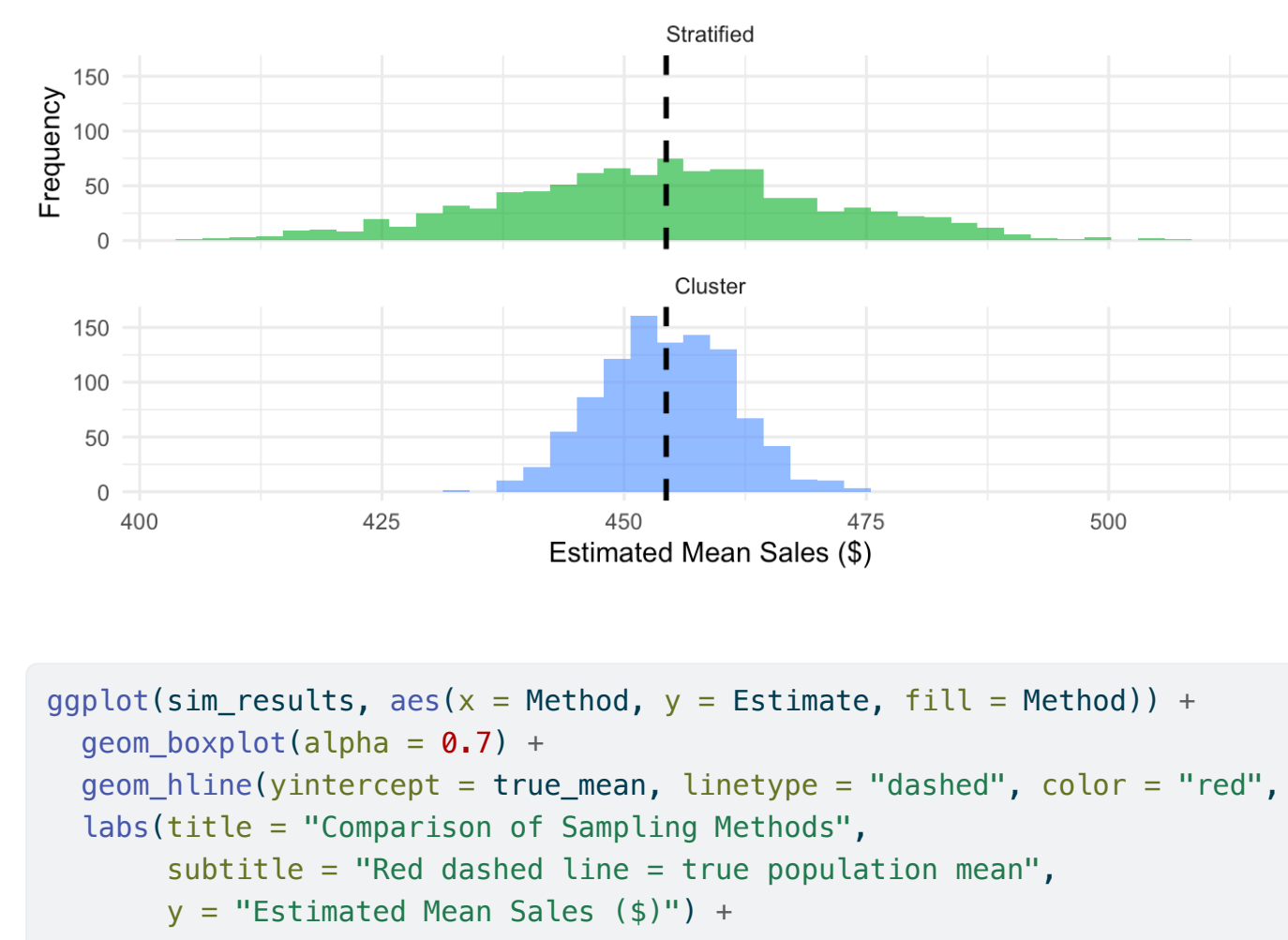
Key Findings:

1. **All methods are approximately unbiased** — bias is negligible relative to the standard error
2. **Cluster has the lowest variance**
3. **SRS has the highest variance**
4. **Stratified sampling has undercoverage** — only 93.8% coverage, indicating standard errors may be underestimated

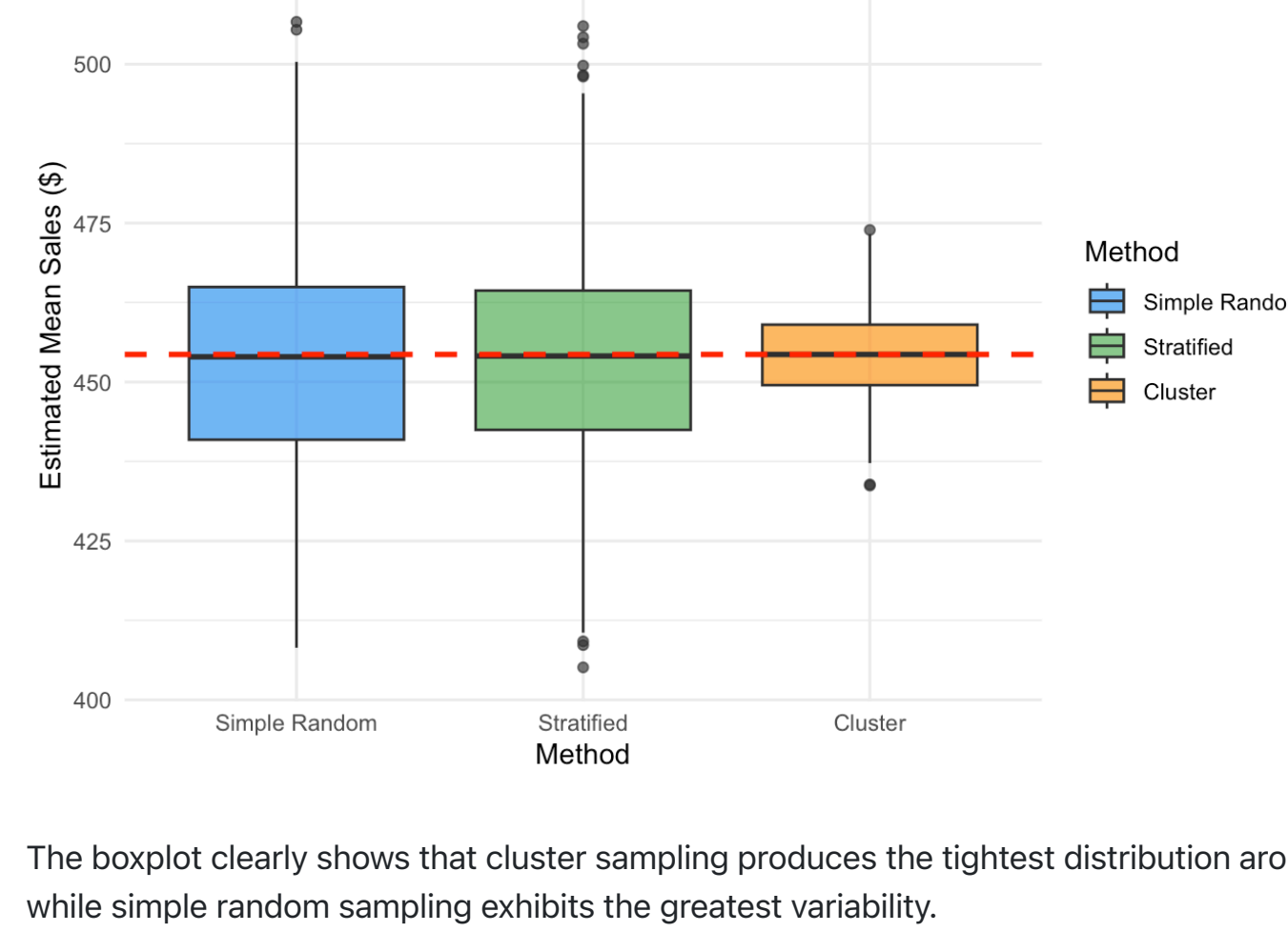
Visual Comparison

```r
# Combine results for plotting
sim_results <- data.frame(
  Estimate = c(results_srs, results_strat, results_cluster),
  Method = rep(c("Simple Random", "Stratified", "Cluster"), each = n_sims)
)

sim_results$Method <- factor(sim_results$Method,
                             levels = c("Simple Random", "Stratified", "Cluster"))

ggplot(sim_results, aes(x = Estimate, fill = Method)) +
  geom_histogram(bins = 40, alpha = 0.7, position = "identity") +
  geom_vline(xintercept = true_mean, linetype = "dashed", size = 1) +
  facet_wrap(~Method, ncol = 1) +
  labs(title = "Sampling Distribution of Mean Estimates (1000 Simulations)",
       subtitle = paste("True population mean = $", round(true_mean, 2)),
       x = "Estimated Mean Sales ($)",
       y = "Frequency") +
  theme_minimal() +
  theme(legend.position = "none")
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
ℹ Please use `linewidth` instead.
```



```r
ggplot(sim_results, aes(x = Method, y = Estimate, fill = Method)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = true_mean, linetype = "dashed", color = "red", size = 1) +
  labs(title = "Comparison of Sampling Methods",
       subtitle = "Red dashed line = true population mean",
       y = "Estimated Mean Sales ($)") +
  theme_minimal() +
  scale_fill_manual(values = c("#2196F3", "#4CAF50", "#FF9800"))
```



The boxplot clearly shows that cluster sampling produces the tightest distribution around the true mean, while simple random sampling exhibits the greatest variability.

Real Data Analysis

We now apply proper estimation formulas to single samples drawn from the population, demonstrating how each method would be used in practice.

Simple Random Sampling

```r
set.seed(789)
n <- 200

# Draw SRS sample
srs_sample <- population[sample(1:N, n, replace = FALSE), ]

# Point estimate
ybar_srs <- mean(srs_sample$sales)

# Variance estimate (with finite population correction)
fpc <- 1 - n/N
var_ybar_srs <- var(srs_sample$sales) / n * fpc
se_srs <- sqrt(var_ybar_srs)

# 95% Confidence interval
ci_srs <- ybar_srs + c(-1, 1) * qt(0.975, df = n-1) * se_srs

cat("=== Simple Random Sampling ===\n")
```

```
=== Simple Random Sampling ===
```

```r
cat("Sample size:", n, "\n")
```

```
Sample size: 200
```

```r
cat("Point estimate: $", round(ybar_srs, 2), "\n")
```

```
Point estimate: $ 447.53
```

```r
cat("Standard error: $", round(se_srs, 2), "\n")
```

```
Standard error: $ 16.07
```

```r
cat("95% CI: ($", round(ci_srs[1], 2), ", $", round(ci_srs[2], 2), ")\n")
```

```
95% CI: ($ 415.84 , $ 479.22 )
```

Stratified Random Sampling

```r
# Proportional allocation
n_strat <- population %>%
  group_by(region) %>%
  summarise(N_h = n()) %>%
  mutate(n_h = round(n * N_h / N))

# Draw stratified sample
strat_sample <- population %>%
  group_by(region) %>%
  sample_n(size = n_strat$n_h[n_strat$region == first(region)]) %>%
  ungroup()

# Calculate stratum statistics
stratum_stats <- strat_sample %>%
  group_by(region) %>%
  summarise(
    n_h = n(),
    ybar_h = mean(sales),
    s2_h = var(sales)
  ) %>%
  left_join(
    population %>% group_by(region) %>% summarise(N_h = n()),
    by = "region"
  ) %>%
  mutate(W_h = N_h / N)

print(stratum_stats)
```

```
# A tibble: 3 × 6
  region   n_h ybar_h   s2_h   N_h   W_h
  <chr>  <int>  <dbl>  <dbl> <int> <dbl>
1 North     59   517. 21537.  1487 0.297
2 South     80   431. 74549.  1992 0.398
3 West      61   429. 43093.  1521 0.304
```

```r
# Stratified estimator
ybar_strat <- sum(stratum_stats$W_h * stratum_stats$ybar_h)

# Variance of stratified estimator
var_ybar_strat <- sum(stratum_stats$W_h^2 * stratum_stats$s2_h / stratum_stats$n_h *
                      (1 - stratum_stats$n_h / stratum_stats$N_h))
se_strat <- sqrt(var_ybar_strat)

# 95% CI (using normal approximation)
ci_strat <- ybar_strat + c(-1, 1) * 1.96 * se_strat

cat("\n=== Stratified Random Sampling ===\n")
```

```

=== Stratified Random Sampling ===
```

```r
cat("Total sample size:", sum(stratum_stats$n_h), "\n")
```

```
Total sample size: 200
```

```r
cat("Point estimate: $", round(ybar_strat, 2), "\n")
```

```
Point estimate: $ 455.88
```

```r
cat("Standard error: $", round(se_strat, 2), "\n")
```

```
Standard error: $ 15.35
```

```r
cat("95% CI: ($", round(ci_strat[1], 2), ", $", round(ci_strat[2], 2), ")\n")
```

```
95% CI: ($ 425.79 , $ 485.97 )
```

Cluster Sampling

```r
# Select random clusters (stores)
n_clusters <- 10
selected_stores <- sample(unique(population$store_id), n_clusters)
cluster_sample <- population[population$store_id %in% selected_stores, ]

# Cluster statistics
cluster_stats <- cluster_sample %>%
  group_by(store_id) %>%
  summarise(
    M_i = n(),
    t_i = sum(sales),
    ybar_i = mean(sales)
  )

print(cluster_stats)
```

```
# A tibble: 10 × 4
   store_id   M_i    t_i ybar_i
      <int> <int>  <dbl>  <dbl>
 1       16    86 39976.   465.
 2       20    95 40327.   424.
 3       22   107 49514.   463.
 4       26   101 46551.   461.
 5       29   105 44431.   423.
 6       35    95 46302.   487.
 7       41   115 55017.   478.
 8       42    91 38631.   425.
 9       44    98 44593.   455.
10       46    89 38961.   438.
```

```r
# Ratio estimator for cluster sampling
M_total <- sum(cluster_stats$M_i)
ybar_cluster <- sum(cluster_stats$t_i) / M_total

# Variance estimation for cluster sampling
m <- n_clusters  # number of clusters sampled
M <- 50  # total number of clusters

# Between-cluster variance
cluster_stats <- cluster_stats %>%
  mutate(d_i = t_i - ybar_cluster * M_i)

s2_clusters <- sum(cluster_stats$d_i^2) / (m - 1)
var_ybar_cluster <- (1 - m/M) * s2_clusters / (m * mean(cluster_stats$M_i)^2)
se_cluster <- sqrt(var_ybar_cluster)

# 95% CI
ci_cluster <- ybar_cluster + c(-1, 1) * qt(0.975, df = m-1) * se_cluster

cat("\n=== Cluster Sampling ===\n")
```

```

=== Cluster Sampling ===
```

```r
cat("Number of clusters:", n_clusters, "\n")
```

```
Number of clusters: 10
```

```r
cat("Total observations:", nrow(cluster_sample), "\n")
```

```
Total observations: 982
```

```r
cat("Point estimate: $", round(ybar_cluster, 2), "\n")
```

```
Point estimate: $ 452.45
```

```r
cat("Standard error: $", round(se_cluster, 2), "\n")
```

```
Standard error: $ 6.63
```

```r
cat("95% CI: ($", round(ci_cluster[1], 2), ", $", round(ci_cluster[2], 2), ")\n")
```

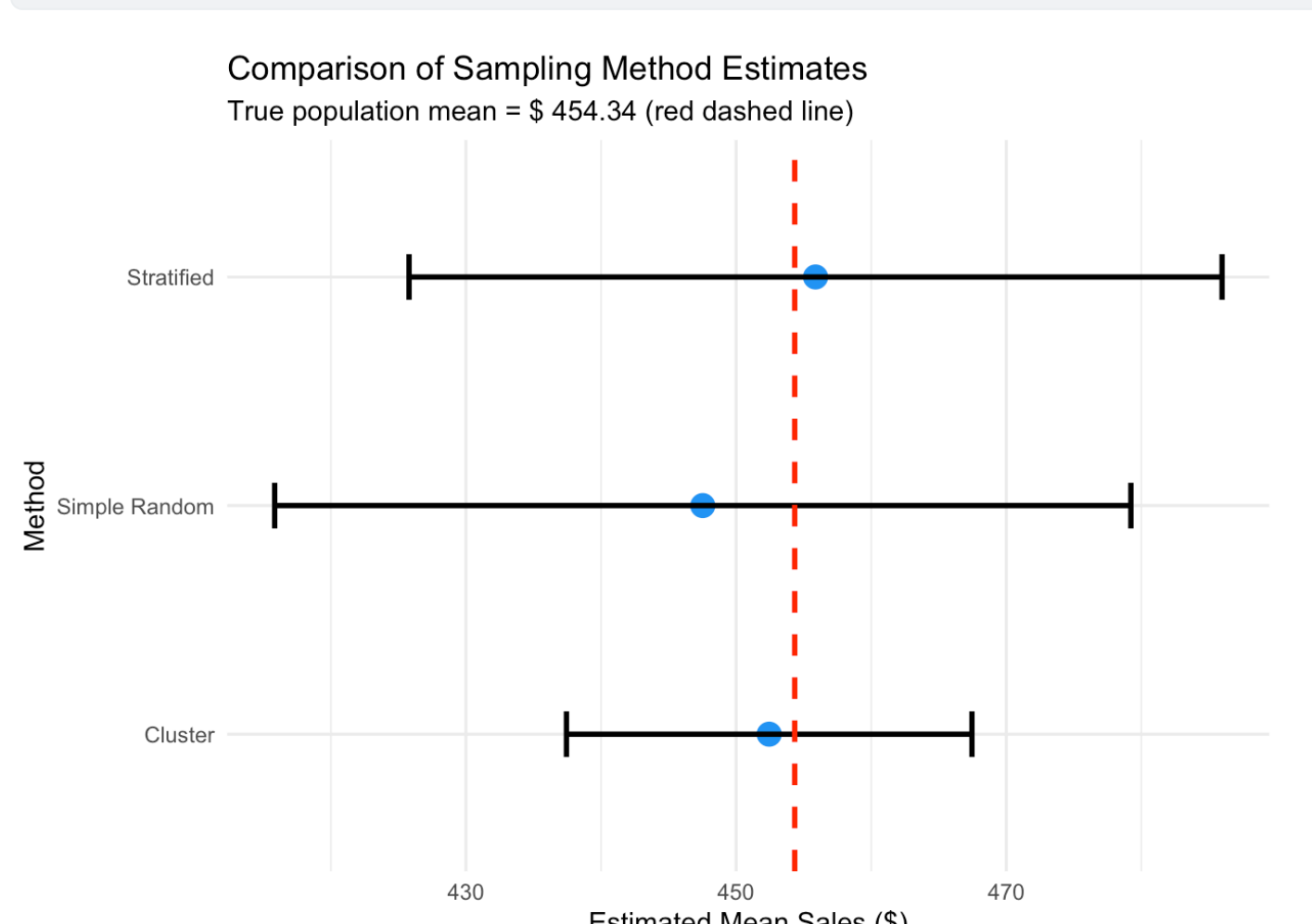
```
95% CI: ($ 437.44 , $ 467.46 )
```

Summary Comparison

```r
comparison <- data.frame(
  Method = c("Simple Random", "Stratified", "Cluster"),
  Sample_Size = c(n, sum(stratum_stats$n_h), nrow(cluster_sample)),
  Estimate = c(ybar_srs, ybar_strat, ybar_cluster),
  SE = c(se_srs, se_strat, se_cluster),
  CI_Lower = c(ci_srs[1], ci_strat[1], ci_cluster[1]),
  CI_Upper = c(ci_srs[2], ci_strat[2], ci_cluster[2]),
  CI_Width = c(diff(ci_srs), diff(ci_strat), diff(ci_cluster))
)

comparison$Contains_True <- comparison$CI_Lower <= true_mean &
                            comparison$CI_Upper >= true_mean

print(comparison)
```

```
         Method Sample_Size Estimate        SE CI_Lower CI_Upper CI_Width
1 Simple Random         200 447.5296 16.070853 415.8386 479.2206 63.38205
2    Stratified         200 455.8791 15.353324 425.7866 485.9716 60.18503
3       Cluster         982 452.4467  6.634738 437.4379 467.4555 30.01764
  Contains_True
1          TRUE
2          TRUE
3          TRUE
```

```r
ggplot(comparison, aes(x = Method, y = Estimate)) +
  geom_point(size = 4, color = "#2196F3") +
  geom_errorbar(aes(ymin = CI_Lower, ymax = CI_Upper), width = 0.2, size = 1) +
  geom_hline(yintercept = true_mean, linetype = "dashed", color = "red", size = 1) +
  labs(title = "Comparison of Sampling Method Estimates",
       subtitle = paste("True population mean = $", round(true_mean, 2), "(red dashed li
       y = "Estimated Mean Sales ($)") +
  theme_minimal() +
  coord_flip()
```



Observations:

- **Cluster sampling** produces the narrowest confidence interval (width = $30.02) despite using the same sample size as SRS
- **Simple random sampling** has the widest confidence interval (width = $63.38) even though it sampled more observations
- All three methods produced confidence intervals that contain the true population mean

Conclusions and Discussion

Summary of Findings

This project evaluated three common sampling methods for estimating average retail sales:

Method	Bias	Variance	RMSE	CI Width	Best For
Simple Random	Negligible	Hightest	17.38	63.38	General use, no prior knowledge
Stratified	Negligible	Moderate	17.06	60.19	Heterogeneous populations
Cluster	Negligible	Lowest	6.74	30.01	Homogeneous clusters

Key Conclusions:

1. Cluster sampling performed best in this analysis, producing the lowest variance and narrowest confidence intervals. This result differs from typical expectations and can be explained by the population structure.
2. Clusters were internally representative. The stores (clusters) in this population contained a diverse mix of sales values, with low between-cluster variance. When clusters are internally heterogeneous but similar to each other, cluster sampling becomes highly efficient.
3. Stratified sampling provided only modest improvement over SRS. This suggests the regional strata did not differ substantially in their mean sales values, limiting the variance reduction benefit of stratification.
4. All methods remained unbiased. Regardless of efficiency differences, each method produced estimates centered on the true population mean.

Theoretical Contex

Standard sampling theory predicts that cluster sampling increases variance due to intraclass correlation — the tendency for units within the same cluster to be more similar than units in different clusters. The intraclass correlation coefficient (ICC) measures this:

$$\rho = \frac{\sigma^2_{between}}{\sigma^2_{between} + \sigma^2_{within}}$$

When rho is close to 0, clusters are essentially random samples of the population, and cluster sampling approaches SRS efficiency. When rho is close to 1, clusters are internally homogeneous, and cluster sampling becomes inefficient.

In this dataset, the low between-cluster variance indicates

$$\rho \approx 0$$

, explaining why cluster sampling outperformed expectations.

Practical Implications

These findings highlight that sampling method performance depends on population structure:

- Retail chains with standardized stores (similar product mix, pricing, demographics) may benefit from cluster sampling because stores represent the overall population well
- Retail chains with regional variation (different climates, cultures, economic conditions) should prefer stratified sampling to capture heterogeneity
- When population structure is unknown, SRS remains a safe default that guarantees unbiased estimation

Limitations

1. Unequal sample sizes — Cluster sampling may have selected more observations than other methods, contributing to lower variance. A fair comparison requires adjusting for effective sample size.
2. Simulated data — The population structure was artificially generated and may not reflect real retail environments.
3. Single population — Results may differ with populations having different cluster structures.

Future Work

Future analyses could address these limitations by:

- Incorporate cost functions to identify optimal sample allocation
- Apply methods to real retail datasets (e.g., from Kaggle or company data)
- Extend to two-stage and multi-stage sampling designs
- Investigate robust variance estimation for complex surveys

References

Cochran, W. G. (1977). *Sampling Techniques* (3rd ed.). Wiley.

Lohr, S. L. (2019). *Sampling: Design and Analysis* (2nd ed.). CRC Press.

Lumley, T. (2010). *Complex Surveys: A Guide to Analysis Using R*. Wiley.

Särndal, C. E., Swensson, B., & Wretman, J. (2003). *Model Assisted Survey Sampling*. Springer.

U.S. Census Bureau. (2023). Annual Retail Trade Survey. Retrieved from census.gov